| | |
|---|---|
| | **Institut Supérieur d'Informatique de Mahdia**<br>**(ISIMa)** |
| **Section : GLSI**<br>**Enseignant : Mme.Waderni Sajida** | **Durée : AU 2024/2025** |
| **TP : Dév.des.app.Mobiles** | |

## Étape 1 : Créer le fichier ProductDetailsScreen.js

Dans votre projet React Native, créez le fichier `ProductDetailsScreen.js`.
Ensuite, ouvrez ce fichier pour y ajouter le code du composant affichant les détails du produit.

### *Étape 2 : Importer les composants nécessaires*

*En haut de votre fichier, importez tous les composants React Native que vous allez utiliser. Dans votre cas, cela inclut :*

***- `StyleSheet` : Pour la gestion des styles.***

***- `Text` et `View` : Pour afficher du texte et organiser la mise en page.***

***- `Image` : Pour afficher les images.***

***- `FlatList` : Pour afficher une liste d'images de manière performante.***

***- `useWindowDimensions` : Pour récupérer les dimensions de l'écran.***

***- `ScrollView` : Pour permettre un défilement vertical du contenu.***

***- `Pressable` : Pour créer un bouton cliquable.***

*Voici le code d'importation :*

```javascript
import { StyleSheet, Text, View, Image, FlatList, useWindowDimensions, ScrollView, Pressable } from 'react-native';
```

***import products from "../data/products";***

***```***

***Étape 3 : Créer le composant d'affichage des détails du produit***

*Dans ce composant, nous affichons les détails du premier produit de la liste (ici `products[0]`). On utilise `useWindowDimensions` pour obtenir la largeur de l'écran, afin de dimensionner les images en conséquence.*

***```javascript***

```
const ProductDetailsScreen = () => {
  const product = products[0];
  const { width } = useWindowDimensions();
return (
    <View>
      <ScrollView>
        <FlatList
          data={product.images}
          renderItem={({ item }) => (
            <Image source={{ uri: item }} style={{ width, aspectRatio: 1 }} />
          )}
          horizontal
          showsHorizontalScrollIndicator={false}
          pagingEnabled
        />
        <View style={{ padding: 20 }}>
          <Text style={styles.title}>{product.name}</Text>
          <Text style={styles.price}>${product.price}</Text>
```

```
            <Text style={styles.description}>{product.description}</Text>
          </View>
        </ScrollView>
        <Pressable style={styles.button}>
          <Text style={styles.buttonText}>Ajouter au panier</Text>
        </Pressable>
      </View>
    );
}
```

### Étape 4 : Préparer les données du produit

Les données sur les produits sont importées depuis le fichier `products.js` situé dans le dossier `data`. Ici, nous supposons que `products` est un tableau d'objets avec les détails de chaque produit (nom, prix, description, images).

### Étape 5 : Utiliser `FlatList` pour afficher les images

Le composant `FlatList` est utilisé pour afficher une liste horizontale d'images. Cela permet d'afficher plusieurs images que l'utilisateur peut faire défiler. L'option `pagingEnabled` permet un défilement fluide d'une image à l'autre.

### Étape 6 : Ajouter les styles

Les styles pour le texte, le bouton, et d'autres éléments sont définis à l'aide de `StyleSheet.create()`. Cela permet de maintenir une structure propre et de réutiliser les styles.

```javascript
const styles = StyleSheet.create({
  title: {
    fontSize: 34,
```

```
    fontWeight: 'bold',
    marginVertical: 10,
  },
  price: {
    fontWeight: '500',
    fontSize: 16,
    letterSpacing: 1.5,
  },
  description: {
    marginVertical: 10,
    fontSize: 8,
    lineHeight: 30,
    fontWeight: '300'
  },
  button: {
    position: 'absolute',
    backgroundColor: 'black',
    bottom: 30,
    width: '90%',
    alignSelf: 'center',
    padding: 20,
    borderRadius: 100,
    alignItems: 'center'
  },
```

```
  buttonText: {
    color: 'white',
    fontWeight: '500',
    fontSize: 16,
  }
});
```

### Étape 7 : Créer le bouton "Ajouter au panier"

Le bouton "Ajouter au panier" est créé à l'aide du composant `Pressable`. Vous pouvez ajouter des actions au clic en fonction de vos besoins (par exemple, ajouter le produit au panier).

```javascript
<Pressable style={styles.button}>
  <Text style={styles.buttonText}>Ajouter au panier</Text>
</Pressable>
```

### Étape 8 : Exporter le composant

Enfin, exportez le composant afin qu'il puisse être utilisé dans votre application.

```javascript
export default ProductDetailsScreen;
```

**Résumé des étapes :**

1. **Ouvrir le fichier du composant.**
2. **Importer les composants nécessaires (`Image`, `FlatList`, etc.).**
3. **Créer le composant principal avec les détails du produit.**

***4. Préparer et importer les données des produits.***

***5. Utiliser `FlatList` pour afficher une liste horizontale d'images.***

***6. Ajouter et organiser les styles avec `StyleSheet`.***

***7. Créer le bouton d'ajout au panier.***

***8. Exporter le composant pour utilisation.***

***Explication de l'utilisation des hooks, en particulier `useWindowDimensions`***

*Les **hooks** dans React permettent d'ajouter des fonctionnalités aux composants fonctionnels (comme la gestion de l'état ou des effets secondaires) sans avoir à utiliser des composants de classe. Ils facilitent la manipulation des états internes, des effets, et des contextes.*

***Qu'est-ce qu'un hook ?***

*Un hook est essentiellement une fonction spéciale qui permet à React d'accéder à certaines fonctionnalités telles que l'état (`useState`), les effets secondaires (`useEffect`), ou les dimensions de la fenêtre (`useWindowDimensions`). Les hooks permettent de rendre les composants plus flexibles et dynamiques.*

***`useWindowDimensions`***

*`useWindowDimensions` est un hook particulier dans React Native qui permet de récupérer les dimensions actuelles de l'écran de l'appareil, c'est-à-dire la **largeur** et la **hauteur** de la fenêtre.*

***Pourquoi utiliser `useWindowDimensions` ?***

*L'utilisation de `useWindowDimensions` permet d'adapter l'affichage de vos composants à la taille de l'écran, ce qui est très utile pour rendre votre application **responsive** (adaptée aux différentes tailles d'écrans). Par exemple, si vous souhaitez ajuster la taille des images ou la disposition des éléments en fonction de l'espace disponible, ce hook est très pratique.*

***# Exemple d'utilisation de `useWindowDimensions`***

*Dans votre code, vous utilisez `useWindowDimensions` pour ajuster la largeur des images en fonction de la taille de la fenêtre :*

***```javascript***

***const { width } = useWindowDimensions();***

```

*Cela permet de récupérer la largeur actuelle de la fenêtre et de l'utiliser dans votre style :*

```***javascript***

***<Image source={{ uri: item }} style={{ width, aspectRatio: 1 }} />***

```

*Ici, l'image est affichée avec une largeur égale à la largeur de l'écran, ce qui assure que l'image s'adapte dynamiquement à la taille de l'appareil.*

*Autres hooks populaires :*

*1. **`useState`** : Gère l'état local dans un composant.*

*2. **`useEffect`** : Gère les effets secondaires, comme les appels d'API ou l'abonnement à un service.*

*3. **`useContext`** : Permet de partager des données entre plusieurs composants sans passer explicitement par les props.*

*Les hooks offrent une manière puissante et intuitive de gérer la logique réactive dans React, simplifiant ainsi la gestion de l'interface utilisateur dynamique et des interactions avec l'utilisateur.*
```